

1. What is hypothesis testing in statistics?

Hypothesis testing is a fundamental statistical method used to make decisions or draw conclusions about a population based on sample data. It provides a structured process to evaluate assumptions or claims, allowing researchers to determine whether there is enough evidence to support a specific belief about a dataset. The primary goal of hypothesis testing is to assess whether an observed effect or difference is genuine or could have occurred by random chance. By evaluating sample data against a predefined assumption, hypothesis testing helps in making objective decisions in scientific research, business analytics, medicine, and social sciences. It is widely used in real-world scenarios such as quality control, clinical trials, A/B testing, product performance evaluation, and survey analysis. Through hypothesis testing, analysts can determine whether to accept or reject a proposed statement using sample information rather than requiring data from the entire population.

2. What is the null hypothesis, and how does it differ from the alternative hypothesis?

The null hypothesis, denoted as H_0 , is a default assumption stating that there is no significant difference, effect, or relationship between variables in the population. It represents a position of neutrality or no change and serves as the basis for statistical testing. The alternative hypothesis, denoted as H_1 or H_a , is the statement that contradicts the null hypothesis and proposes that a significant effect, relationship, or difference does exist. While the null hypothesis assumes status quo, the alternative hypothesis represents what the researcher aims to prove. For example, if a company claims that the average weight of a product is 500g, the null hypothesis would be $H_0: \mu = 500\text{g}$, whereas the alternative hypothesis could be $H_1: \mu \neq 500\text{g}$. Hypothesis testing evaluates sample evidence to determine whether there is enough statistical support to reject the null hypothesis in favour of the alternative. If the evidence is insufficient, the null hypothesis is retained.

3. Explain the significance level in hypothesis testing and its role in deciding the outcome of a test.

The significance level, denoted by α (alpha), is a threshold used to determine the probability of rejecting the null hypothesis when it is actually true. It represents the level of risk a researcher is willing to accept while making a decision. Common values of significance level are 0.05 (5%), 0.01 (1%), and 0.10 (10%). A significance level of 0.05 means there is a 5% chance of committing a Type I error — rejecting a true null hypothesis. In hypothesis testing, the p-value calculated from the test results is compared with the significance level: if the p-value is less than α , the null hypothesis is rejected, indicating that the observed result is statistically significant. Conversely, if the p-value is greater than α , there is not enough evidence to reject the null hypothesis. Therefore, the significance level plays a critical role in decision-making, determining the strength of evidence required to support or reject a claim.

4. What are Type I and Type II errors? Give examples of each.

Answer: Type I and Type II errors are critical concepts in hypothesis testing that describe incorrect decisions made when evaluating sample data. A Type I error occurs when the null hypothesis is true, but it is incorrectly rejected. This is also known as a false positive. For example, if a medical test suggests that a patient has a disease when they are actually healthy, it results in a Type I error. The probability of committing a Type I error is represented by alpha (α), which is the significance level. In contrast, a Type II error occurs when the null hypothesis is false, but it is incorrectly accepted. This is also referred to as a false negative. For instance, if a medical test fails to detect a disease that a patient truly has, it is a Type II error. The probability of this error is denoted by beta (β). Both errors are important in determining the effectiveness of hypothesis testing, and researchers must balance them to minimize incorrect decisions.

5 What is the difference between a Z-test and a T-test? Explain when to use each.

A Z-test and a T-test are statistical methods used to compare sample means and determine whether they significantly differ from a known population mean. The primary difference between them lies in the sample size and whether the population standard deviation is known. A Z-test is used when the sample size is large (typically $n > 30$) and the population standard deviation is known. It assumes that the data follows a normal distribution. Z-tests are commonly used in quality control, population studies, and large-scale surveys. On the other hand, a T-test is used when the sample size is small ($n < 30$) and the population standard deviation is unknown. The T-test uses the sample standard deviation as an estimate. T-tests are widely applied in experiments, clinical trials, and academic research where samples are limited. Both tests help determine whether sample data provides enough evidence to reject the null hypothesis.

6. Write a Python program to generate a binomial distribution with $n=10$ and $p=0.5$, then plot its histogram.

A binomial distribution is a discrete probability distribution that represents the number of successes in a fixed number of independent Bernoulli trials, where each trial has only two possible outcomes: success or failure. The probability of success remains constant across all trials. In this problem, we generate a binomial distribution using parameters $n = 10$ (number of trials) and $p = 0.5$ (probability of success per trial). We then simulate 1000 random outcomes based on the binomial distribution and visualize the distribution using a histogram.

The histogram allows us to observe how frequently each number of successes occurs in repeated trials. Since the probability of success is 0.5, the distribution appears symmetric around its mean (which is $np = 10 \times 0.5 = 5$). Most simulated values cluster around the center (4–6), demonstrating the typical bell-like shape of a binomial distribution.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
```

```

data_binom = np.random.binomial(n=10, p=0.5, size=1000)

plt.hist(data_binom)

plt.title("Binomial Distribution (n=10, p=0.5)")

plt.xlabel("Number of Successes")

plt.ylabel("Frequency")

plt.show()

```

7. Implement hypothesis testing using Z-statistics for a sample dataset in Python

We will test the null hypothesis $H_0: \mu = 50.0$ against the two-tailed alternative $H_a: \mu \neq 50.0$ using the supplied sample. Because the sample size $n = 36$ is reasonably large (≥ 30), we use the z-statistic approximation. If the population standard deviation were known we would use it directly; here we use the sample standard deviation and treat the z-approximation as acceptable because of sample size.

Python code :

```

import numpy as np

from scipy import stats

import matplotlib.pyplot as plt


sample_data = np.array([
    49.1, 50.2, 51.0, 48.7, 50.5, 49.8, 50.3, 50.7, 50.2, 49.6,
    50.1, 49.9, 50.8, 50.4, 48.9, 50.6, 50.0, 49.7, 50.2, 49.5,
    50.1, 50.3, 50.4, 50.5, 50.0, 50.7, 49.3, 49.8, 50.2, 50.9,
    50.3, 50.4, 50.0, 49.7, 50.5, 49.9
])

n = len(sample_data)

xbar = sample_data.mean()

s = sample_data.std(ddof=1)      # sample SD

mu0 = 50.0

z_stat = (xbar - mu0) / (s / np.sqrt(n))

p_value = 2 * (1 - stats.norm.cdf(abs(z_stat))) # two-tailed p-value

```

```

print("n =", n)
print("Sample mean =", xbar)
print("Sample sd  =", s)
print("Z-statistic =", z_stat)
print("Two-tailed p-value =", p_value)
plt.hist(sample_data, bins=10, edgecolor='black')
plt.axvline(x=xbar, color='orange', linestyle='--', label=f"Sample mean = {xbar:.3f}")
plt.axvline(x=mu0, color='green', linestyle=':', label=f"Null mean = {mu0}")
plt.title("Sample Data Distribution (Q7)")
plt.legend()
plt.show()

```

Numeric results:

- $n = 36$
- Sample mean $\bar{x} = 50.08888888888889$
- Sample standard deviation $s = 0.5365379910807955$
- Z-statistic = **0.9940271559503017**
- Two-tailed p-value = **0.3202096468890012**

8. Simulate normal data and calculate 95% confidence interval for its mean

We simulate $n = 100$ observations from a Normal distribution with true mean $\mu = 5.0$ and true standard deviation $\sigma = 2.0$. We compute the sample mean, sample standard deviation, standard error, and the 95% confidence interval for the mean using the normal critical value $z_{0.975} \approx 1.96$ (appropriate for large samples).

Python code:

```

import numpy as np

from scipy import stats

import matplotlib.pyplot as plt

np.random.seed(0)    # for reproducibility

```

```

n = 100
mu_true = 5.0
sigma_true = 2.0
data = np.random.normal(loc=mu_true, scale=sigma_true, size=n)
sample_mean = data.mean()
sample_std = data.std(ddof=1)
se = sample_std / (n**0.5)
z_crit = stats.norm.ppf(0.975) # ~1.96
ci_lower = sample_mean - z_crit * se
ci_upper = sample_mean + z_crit * se
print("n =", n)
print("Sample mean =", sample_mean)
print("Sample sd =", sample_std)
print("95% CI = [{:.6f}, {:.6f}]" .format(ci_lower, ci_upper))
plt.hist(data, bins=15, edgecolor='black')
plt.axvline(sample_mean, color='red', linestyle='--', label=f"Mean = {sample_mean:.3f}")
plt.axvline(ci_lower, color='blue', linestyle=':', label=f"95% CI lower = {ci_lower:.3f}")
plt.axvline(ci_upper, color='blue', linestyle=':', label=f"95% CI upper = {ci_upper:.3f}")
plt.title("Simulated Normal Data and 95% CI (Q8)")
plt.legend()
plt.show()

```

Numeric results:

- Sample mean = **5.11961603106897**
- Sample standard deviation = **2.0259195385370035**
- 95% confidence interval for the mean = **[4.722543097958117, 5.516688964179823]**

9. Function to calculate Z-scores and visualize standardized data

A Z-score (standard score) indicates how many standard deviations an observation is from the sample mean. For a value x , the Z-score is $z = \frac{x - \bar{x}}{s}$, where $\bar{x}$ is the sample mean and s is the sample standard

deviation. Z-scores standardize different datasets and make them comparable; they are also useful for detecting outliers.

Python code:

```
import numpy as np
import matplotlib.pyplot as plt

def compute_z_scores(data):
    data = np.array(data, dtype=float)
    mean = data.mean()
    sd = data.std(ddof=1)
    z = (data - mean) / sd
    return z, mean, sd

sample_data = np.array([
    49.1, 50.2, 51.0, 48.7, 50.5, 49.8, 50.3, 50.7, 50.2, 49.6,
    50.1, 49.9, 50.8, 50.4, 48.9, 50.6, 50.0, 49.7, 50.2, 49.5,
    50.1, 50.3, 50.4, 50.5, 50.0, 50.7, 49.3, 49.8, 50.2, 50.9,
    50.3, 50.4, 50.0, 49.7, 50.5, 49.9
])

z_scores, mean_val, sd_val = compute_z_scores(sample_data)
print("Mean =", mean_val)
print("Sample SD =", sd_val)
print("First 10 Z-scores:", z_scores[:10])
plt.hist(z_scores, bins=12, edgecolor='black')
plt.title("Histogram of Z-scores (Q9)")
plt.xlabel("Z-score")
plt.ylabel("Frequency")
plt.axvline(0, color='red', linestyle='--', label='Mean (0)')
plt.legend()
plt.show()
```

Numeric results:

- Sample mean = **50.0888888888889**
- Sample standard deviation = **0.5365379910807955**
- First 10 Z-scores (approx):
- [-1.84309202, 0.20708899, 1.69812972, -2.58861239, 0.76622927, -0.53843138, 0.39346908, 1.13898945, 0.20708899, -0.91119156]